TrialCheck

Interview Defense Guide — v0.2.0

Platform-agnostic A/B experiment readout auditor

1. What Is TrialCheck?

TrialCheck audits completed A/B experiment readouts. It accepts an ExperimentSummary (assignment counts, metric data, optional guardrails and pre-period covariates) and runs seven checks grouped into three categories: randomization integrity, statistical validity, and practical readiness. It returns a TrialReport with per-check PASS/WARN/FAIL/INSUFFICIENT_INPUT verdicts, an overall status, and a human-readable interpretation.

It is decision support, not a decision-maker. It does not run experiments, assign users, perform CUPED, correct for multiple comparisons, or make ship decisions.

Check	Group	Method	Fires when
SRM	Randomization	chi² df=1, erfc(√x/2)	Observed split deviates from planned ratio
Pre-period balance	Randomization	SMD per covariate, pooled SD	Any covariate SMD ≥ 0.1 pre-treatment
Primary metric	Statistical	Two-proportion z-test, pooled SE under H■	p ≤ alpha or missing counts
Continuous metric	Statistical	Welch t-test, Welch–Satterthwaite df	p > alpha for mean comparison
Peeking risk	Statistical	Duration ratio + interim look count	Test stopped before planned duration
Practical significance	Practical	Observed lift vs business threshold	Lift real but negligible
Guardrail movement	Practical	Direction check + tolerance band	Guardrail moves bad direction past tolerance

2. Statistical Implementations

2.1 SRM Check

Under H■ (no assignment drift), the number of units assigned to each variant follows a multinomial distribution with expected proportions equal to the planned split. For a two-variant test, this reduces to a chi-square test with df=1. The p-value is computed exactly as erfc(√(χ²/2)), where erfc is the complementary error function from Python's math module — no scipy dependency. SRM fires FAIL (not WARN) because assignment drift invalidates all downstream statistical inference: if users were non-randomly allocated, the treatment effect estimate is confounded.

2.2 Two-Proportion Z-Test

For binary metrics (conversion rate, click-through), the test statistic is z = (p■■ – p■■) / SE, where SE is computed under H■ using the pooled proportion p■ = (x■ + x■)/(n■ + n■). Using pooled SE under H■ is the correct frequentist formulation: under the null, both variants share the same true proportion, so pooling is appropriate. Using unpooled SE (the Wald interval formula) would be anti-conservative.

2.3 Welch's T-Test

For continuous metrics, the test uses Welch's t-test rather than Student's t-test. Student's t-test assumes equal variances — but treatment effects frequently change variance (e.g., a new feature increases mean revenue but also variance among high-value users). Welch's test uses separate variance estimates per arm and the Welch–Satterthwaite equation for degrees of freedom: df = (s■²/n■ + s■²/n■)² / [(s■²/n■)²/(n■–1) + (s■²/n■)²/(n■–1)]. On Python 3.12+, the t-distribution CDF is computed exactly via math.betainc; on 3.10/3.11, the normal approximation via math.erfc is used (valid when df > 30).

2.4 Pre-Period Balance: Standardised Mean Difference

SMD = (mean_treatment – mean_control) / pooled_SD, where pooled_SD = √((s■² + s■²)/2). SMD ≥ 0.1 triggers WARN. This threshold comes from Austin (2011) on propensity score matching: below 0.1 is considered adequate balance for observational studies; in a randomised experiment, any non-trivial SMD suggests assignment went wrong.

3. Design Decisions

3.1 SRM is always FAIL, not WARN

Every other check in TrialCheck returns WARN because the finding requires context to interpret. SRM is different: if assignment drifted, the treatment group and control group are systematically different from the start. All downstream p-values, lift estimates, and guardrail comparisons are measuring a confounded comparison, not a clean experiment. There is no scenario where a significant SRM is acceptable to ignore.

3.2 MDE Context is not retrospective power

The MDE check compares observed lift to the planned MDE. It is explicitly framed as 'is the observed effect below the sensitivity the test was designed to detect?' — not as 'did this experiment have enough power?' Post-hoc power calculation is statistically invalid (Hoenig & Heisey, 2001): observed power is a monotone transformation of the observed p-value and adds no information. TrialCheck never computes post-hoc power.

3.3 No multiple comparison correction for guardrails

The current version does not apply Bonferroni or Holm correction across guardrail metrics. This is a deliberate v0 scope decision: adding correction would require defining the family of hypotheses (primary + all guardrails? guardrails only?), and different teams have different conventions. Correction is a documented roadmap item.

3.4 Zero dependencies

All statistical computations use only Python's standard math module (erfc, sqrt, betainc on 3.12+). This makes TrialCheck installable in any environment without scipy, statsmodels, or numpy version conflicts — critical for use as a CI dependency or pre-commit hook.

4. Hard Interview Questions

Q1: Your SRM check uses df=1. What if the test had three variants?

A: df=1 is correct for two-variant tests: one observed cell determines the other given fixed n. For k variants, the chi-square statistic sums $(O_i - E_i)^2/E_i$ over all k cells and df = k–1. The current implementation is limited to two-variant tests; multi-variant SRM is a documented roadmap item. Applying df=1 to a 3-variant test would under-estimate the p-value and produce false-negative SRMs on balanced-looking splits that are actually drifted.

Q2: Why use a two-proportion z-test instead of Fisher's exact test for binary metrics?

A: Fisher's exact test is preferable for small n (typically n < 1000 per arm) because it doesn't rely on the normal approximation. For internet-scale A/B tests with tens of thousands of users per arm, the normal approximation is excellent and the z-test is computationally trivial. Fisher's exact becomes numerically challenging at large n due to hypergeometric distribution computation. TrialCheck targets the common case; Fisher's exact is a documented extension for small-n clinical or UX tests.

Q3: The peeking risk check uses duration ratio. Is that a valid proxy for alpha inflation?

A: Duration ratio is a heuristic, not a rigorous bound. Exact alpha inflation from peeking depends on the sequence of look times, the correlation structure, and the specific sequential testing rule violated. A test stopped at 40% of planned duration with 0 interim looks could be fine (if the team never looked at interim results) or severely inflated (if they checked daily). The check returns WARN, not FAIL, because duration ratio alone cannot distinguish these cases. The recommendation is to apply a sequential testing correction (alpha spending, mSPRT) retrospectively or to extend the test.

Q4: Why Welch's t-test rather than Mann-Whitney U for continuous metrics?

A: Mann-Whitney U tests the null that $P(X > Y) = 0.5$ — a stochastic dominance hypothesis, not a mean difference. For business metrics like revenue/user, the mean is the quantity that matters for P&L; decisions, not stochastic ordering. Welch's t-test directly tests the mean difference. Additionally, Welch's t-test produces an interpretable effect size (mean difference with confidence interval) while Mann-Whitney produces a rank-biserial correlation that is harder to translate into business impact.

Q5: Post-hoc power is invalid — you mentioned Hoenig & Heisey. What's the argument?

A: Given observed data, observed power = f(observed p-value). If p = 0.04 (barely significant), observed power will appear low. If p = 0.001, it will appear high. The two numbers carry identical information. Computing 'we had 43% power, so the null result is unsurprising' is circular: you're restating the p-value in power language, which sounds more authoritative but adds no new content. TrialCheck's MDE check avoids this by comparing the observed effect to the planned MDE — a forward-looking design question, not a backward-looking power calculation.

Q6: What does TrialCheck miss that a complete experiment review would include?

A: Several things. Network effects / SUTVA violations — if treatment users affect control users (e.g., social features, shared resources), the stable unit treatment value assumption is violated and the estimated effect is biased. Novelty and primacy effects — short-duration tests often capture user curiosity about a new feature, not steady-state behaviour. Segment heterogeneity — a positive overall lift can mask a negative effect on a key segment. Multiple comparisons across primary and secondary metrics. CUPED / variance reduction. All of these require either more data, more config, or domain judgment that can't be automated.

Q7: The SMD threshold of 0.1 — where does that come from and is it appropriate here?

A: The 0.1 threshold originates in observational study methodology (Austin 2011, propensity score research). In a randomised experiment, any non-trivial SMD before treatment is a signal that assignment went wrong — so 0.1 is arguably generous. Stricter reviewers use 0.05. The threshold is exposed as a configurable parameter in ExperimentSummary so teams can tighten it. The default of 0.1 is deliberately lenient to avoid WARN noise on large experiments where small SMDs are expected by chance.

Q8: What if a guardrail metric improves? Should that be PASS?

A: Improvement in a bad direction is impossible by definition — bad_direction is explicitly declared in GuardrailMetric (e.g., bad_direction='decrease' for revenue). If the metric moves in the good direction, the check returns PASS regardless of magnitude. If it moves in the bad direction within the tolerance band, it returns WARN. Beyond tolerance, it returns FAIL. This design means a treatment that improves all guardrails doesn't generate spurious warnings.

Q9: Could two experiments both pass TrialCheck but one be more trustworthy than the other?

A: Yes. TrialCheck produces a binary PASS/WARN/FAIL per check — it doesn't rank confidence across experiments. An experiment with 500,000 users per arm that passes all checks is more trustworthy than one with 2,000 users per arm that also passes, even though both produce identical report statuses. Effect size confidence intervals, statistical power margins, and experiment duration relative to business cycle length are not captured in the current check set. TrialCheck is a necessary condition for trustworthy readouts, not a sufficient one.

Q10: If a team runs TrialCheck in CI and the report is FAIL, should they auto-block shipping?

A: SRM FAIL and guardrail FAIL are strong candidates for automatic gates because they represent structural problems (assignment drift) or definitive harm (guardrail breach). WARN findings (peeking, practical significance, MDE) require human judgment: a team might consciously accept a WARN and document the reason. The recommended pattern is: auto-block on FAIL, require human sign-off with a comment on WARN, auto-pass on PASS. TrialCheck's JSON output makes this CI integration straightforward.

5. Claim Boundary

TrialCheck does not:

• Run experiments or assign users to variants

• Prove causal validity of an effect

• Perform CUPED, variance reduction, or pre-experiment covariate adjustment

• Apply multiple comparison correction across metrics

• Handle network effects, SUTVA violations, or novelty effects

• Make automatic ship/no-ship decisions

TrialCheck does:

• Systematically check seven known readout risks in one call

• Return structured, machine-parseable verdicts with recommendations

• Surface the same questions a senior DS would ask at readout review

• Produce a documented, reproducible audit record attachable to a decision log

6. Files Index

File	Purpose
src/trialcheck/checks.py	7 check implementations
src/trialcheck/stats.py	welch_t_test, normal_two_sided_pvalue, chi_square_pvalue
src/trialcheck/auditor.py	TrialCheck.run() — orchestrates all checks
src/trialcheck/models.py	ExperimentSummary, VariantSummary, GuardrailMetric, PrePeriodCovariate
tests/test_trialcheck.py	17 unit tests across 3 test classes
scripts/generate_demo_reports.py	4 canonical scenarios: clean_pass, srm_fail, peeking_warn, guardrail_harm
README.md	Badges, LR arch diagram with 3 groups, About, check table, quickstart
CHANGELOG.md	v0.1.0 and v0.2.0 release notes
.github/workflows/ci.yml	Python 3.10/3.11/3.12 CI matrix
.github/workflows/publish.yml	OIDC PyPI trusted publisher on release

Resume-safe claim: Built TrialCheck, a platform-agnostic A/B experiment readout auditor that checks completed experiment summaries for sample-ratio mismatch (chi-square), peeking risk, MDE context, practical and statistical significance (two-proportion z-test and Welch's t-test), guardrail movement, and pre-period covariate imbalance (SMD), producing JSON/Markdown/HTML audit reports with explicit decision caveats. Zero dependencies. 17 tests. 4 canonical demo scenarios.